

مشروع «مجلس» (Majlis)

منصة لإدارة وتنسيق الندوات والدروس العلمية وحجز المقاعد

JWT

Redis

TypeORM

PostgreSQL

NestJS

1. نظرة عامة على المشروع (Project Vision)

Majlis هي منصة لإدارة وتنسيق الندوات والدروس العلمية، وتسهيل عملية حجز المقاعد للمستخدمين، مع التعامل مع أوقات الذروة وازدحام الحجوزات في لحظة إطلاق الندوة.

2. أدوار المستخدمين والصلاحيات (RBAC - User Roles)

المستخدم / الحاضر ATTENDEE

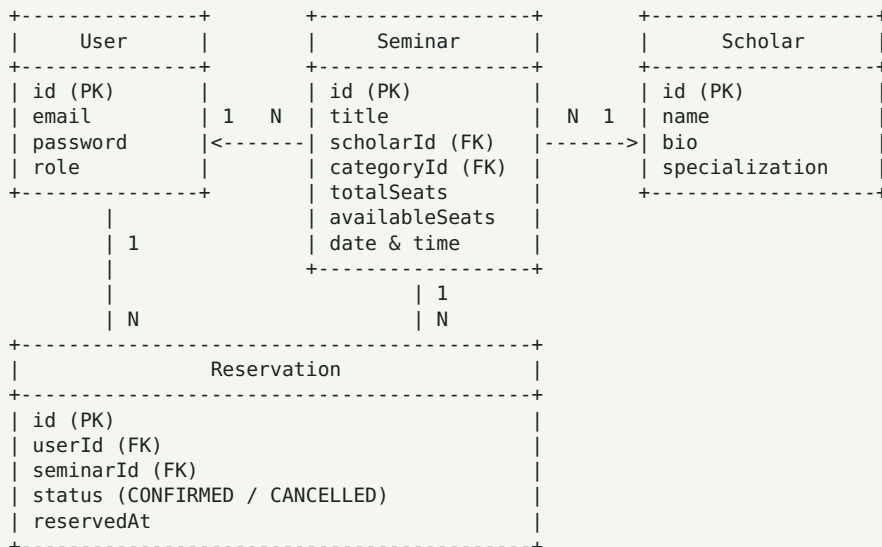
- تصفح الندوات والبحث حسب المحاضر أو التصنيف.
- حجز مقعد في ندوة أو إلغاء الحجز.
- عرض تذاكر الحجز الخاصة به.

المشرف العام ADMIN

- إدارة المحاضرين والعلماء (Scholars).
- إنشاء وتعديل وإلغاء الندوات (Seminars).
- الاطلاع على تقارير الحجوزات ونسبة الإشغال.

3. معمارية قاعدة البيانات والعلاقات (Database Schema & Entities)

سنستخدم PostgreSQL مع TypeORM. جاء اختيار PostgreSQL لأنه يضمن سلامة البيانات بفضل خاصية ACID Compliance، ولقدرته القوية على التعامل مع المعاملات المتزامنة (Concurrent Transactions).



العلاقات التقنية (Entity Relations)

User 1 : N Reservation – المستخدم الواحد يمكنه إجراء عدة حجوزات.

Seminar 1 : N Reservation – الندوة الواحدة تضم عدة حجوزات.

Scholar 1 : N Seminar – المحاضر الواحد يمكنه إلقاء عدة ندوات.

Category 1 : N Seminar – التصنيف الواحد (فقه، تفسير...) يضم عدة ندوات.

4. مشاكل النظام المتقدمة وحلولها (System Design & Trade-offs)

المشكلة 1: الحجز المزدوج وضغط الذروة (Race Conditions & Overbooking)

التحدي

إذا كان المتبقي مقعد واحد فقط في ندوة للشيخ، وحاول 50 مستخدماً الضغط على زر «حجز» في اللحظة نفسها، فقد ينتهي الأمر بحجز أكثر من السعة المتاحة (Overbooking).

الحل

- **Pessimistic Locking / Database Transactions:** عند إجراء عملية الحجز نفتح Transaction في PostgreSQL ونضع قفلاً على صف الندوة `SELECT ... FOR UPDATE` للقراءة والكتابة حتى تنتهي عملية التأكيد وخصم المقعد `availableSeats - 1`.
- **Unique Constraint:** وضع شرط فريد على جدول الحجوزات يتكوّن من `(userId, seminarId)` لمنع المستخدم نفسه من حجز أكثر من مقعد في الندوة الواحدة.

المشكلة 2: الاختناق في الاستعلامات المكثرة (Read-Heavy Bottlenecks)

التحدي

آلاف المستخدمين يتصفحون قائمة الندوات القادمة، مما يسبب ضغطاً كبيراً على استعلامات القراءة في قاعدة البيانات (Database Read Queries).

الحل

- **Redis Caching:** تخزين قائمة الندوات الشائعة أو القادمة مؤقتاً في ذاكرة Redis لتخفيف الحمل على PostgreSQL.

المشكلة 3: حماية النقاط الحساسة (API Rate Limiting)

التحدي

هجمات Brute-force على تسجيل الدخول، أو إغراق ال API بطلبات حجز وهمية.

الحل

• تطبيق `@nestjs/throttler` لتحديد عدد الطلبات المسموح بها لكل IP (مثلاً 10 طلبات في الدقيقة لمنع الإغراق).

5. الهيكلة المعمارية للمشروع في NestJS (Folder Structure)

سنتبّع نمط Modular Architecture المعتاد في NestJS لضمان فصل المسؤوليات:

```
src/
├── common/                # Guards, Interceptors, Decorators Shared
│   ├── decorators/        # Custom Decorators (@GetUser, @Roles)
│   ├── guards/            # JwtAuthGuard, RolesGuard
│   └── filters/           # Global HttpExceptionsFilter
├── config/                # Database, JWT & App configurations
├── modules/
│   ├── auth/              # Login, Register, JWT, Hashing
│   ├── users/             # User Service, Entity, DTOs
│   ├── scholars/          # Scholar Management
│   ├── seminars/          # Seminar Management & Search
│   └── reservations/      # Booking Logic, Locking & Transactions
└── main.ts                # Entry Point (ValidationPipe, Swagger)
```